Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	**192524358**
Student Name	**Shaik. Abdul Razak**

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])          // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

SCALABILITY AUDIT OF SHOPSPHERE

(a) Understand the Problem

M1 – Duplicate-Listing Detector:

- The input size is n products.
- The dominant operation is comparing the title and seller of each pair of products in the nested loops.
- It performs $n(n-1)/2$ comparisons, so its time complexity is **$\Theta(n^2)$.**

M2 – API Rate Limiter:

- The input size is n requests.
- The basic operation is Audit (Request[j]).
- The inner loop runs 1, 2, 4, 8, … times.
- The total is a geometric series less than 2n, so the time complexity is $\Theta(n)$, not **$\Theta(n \log n)$.**

M3 – Product Search S1:

- The input size is n catalogue entries.
- The basic operation is comparing the target with each product during a sequential scan.
- It takes $\Theta(1)$ in the best case and $\Theta(n)$ in the worst case.

M3 – Product Search S2:

- The input size is n entries in a sorted index.
- The basic operation is comparing the target with the middle entry.
- Each comparison removes half of the remaining entries, giving **$\Theta(\log n)$** time.

M4 – Recommendation Engine:

- The input size is n customer–product records.
- The current recurrence is **$T(n)=4T(n/2)+n$** and has **$\Theta(n^2)$** complexity.
- The proposed recurrence **$T(n)=2T(n/2)+n$** has **$\Theta(n \log n)$** complexity.

M5 – Sales Analytics Engine:

- The input size is n sales records.
- The recurrence is **$T(n)=2T(n/2)+n \log_2 n$.**
- Its time complexity is **$\Theta(n(\log n)^2)$.**

(b) Analyse the Iterative Modules

M1 – Exact number of comparisons

- For $i = 0$, the inner loop makes $n-1$ comparisons.
- For $i = 1$, it makes $n-2$ comparisons.
- This continues until the final pass makes 1 comparison.

$C(n) = (n-1)+(n-2)+\ldots+2+1$

$C(n) = 1+2+\ldots+(n-1) = n(n-1)/2$.

- Therefore, the time complexity is $\Theta(n^2)$.
- Auxiliary space is $\Theta(1)$ if duplicate marks are stored in the existing product records.
- If a separate list of duplicate indices is stored, that output list can require $O(n)$ space.

M2 – Exact number of Audit operations

- The values of i are 1, 2, 4, 8, …, 2^k, where 2^k is the largest power of two less than n.
- A(n) = $1+2+4+\ldots+2^k = 2^{k+1}-1$.
- For n > 1, A(n) = $2^{\lceil \log_2 n \rceil}-1$.
- Since $n \leq 2^{\lceil \log_2 n \rceil} < 2n$, A(n) is Θ(n).
- Therefore, the claim Θ(n log n) is incorrect.
- For $n=1{,}024=2^{10}$, the audit count is $1+2+4+\ldots+512 = 2^{10}-1 = 1{,}023$ operations.

(c) Recurrence by Substitution: Search Version S2

T(n) = T(n/2) + 1, with T(1) = 1

Backward substitution:

- **Step 1**: T(n) = T(n/2) + 1
- **Step 2:** replace T(n/2) = T(n/4) + 1
- T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2
- **Step 3:** replace T(n/4) = T(n/8) + 1
- T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3
- **Pattern after k steps**: T(n) = T(n/2^k) + k
- Stop when n/2^k = 1, which gives 2^k = n, so $k = \log_2 n$.
- **$T(n) = T(1) + \log_2 n = 1 + \log_2 n$**

Exact closed form: $T(n) = \log_2 n + 1$ (for n a power of 2)

Hence, the time complexity is Θ(log n). Iterative binary search uses Θ(1) auxiliary space. Recursive binary search uses Θ(log n) call-stack space in the worst case.

Implementation	Time complexity	Space complexity
Iterative	Θ(log n)	Θ(1) (only low, high, mid are stored)
Recursive	Θ(log n)	Θ(log n) (the call stack is log n levels deep)

(d) Apply the Master Theorem

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a):

- **Case 1:** f(n) is polynomially smaller, so T(n) = Θ(n^(log_b a)).
- **Case 2:** f(n) = Θ(n^(log_b a)), so T(n) = Θ(n^(log_b a) · log n).
- **Case 3:** f(n) is polynomially larger (and the regularity condition holds), so T(n) = Θ(f(n)).

M4 – Current design $T(n)=4T(n/2)+n$.

- Here a=4, b=2, and f(n)=n.
- n^(log_b a)=$n^{(\log_2 4)}=n^2$.
- Since f(n)=n=O(n^(2−1)), f(n) is polynomially smaller than n^2.
- Master Theorem Case 1 applies.
- **Result:** $T(n)=\Theta(n^2)$.
- If the input size doubles, the dominant work increases by approximately $2^2=4$ times.

M4 – Proposed redesign T(n)=2T(n/2)+n.

- Here a=2, b=2, and f(n)=n.
- n^(log_b a)=n^($\log_2$2)=n.
- Since f(n)=Θ(n^(log_b a)),
- Master Theorem Case 2 applies.
- **Result:** T(n)=Θ(n log n). When n doubles, the work increases by approximately 2($\log_2$n+1)/$\log_2$n, which approaches 2 for large n.

M5 – Sales analytics engine T(n)=2T(n/2)+n $\log_2$n.

- Here a=2, b=2, and n^(log_b a)=n.
- The function f(n)=n log n is larger than n by a logarithmic factor, so the three basic Master Theorem cases do not directly apply.
- Use an extended Master Theorem or recursion tree.
- At level i, there are 2^i subproblems, each of size $n/2^i$.
- The work at that level is $2^i \times (n/2^i) \log_2(n/2^i) = n(\log_2 n - i)$.
- For $n=2^k$, the total internal work is $n[k+(k-1)+\ldots+1] = nk(k+1)/2$. Leaf work is Θ(n).
- **Result: T(n)=Θ(n(log n)²).**

Check using a recursion tree:

- At level i there are 2^i sub-problems, each of size n/2^i.
- Cost of one sub-problem = (n/2^i) · log(n/2^i), so the cost of level i = n · (log n − i).
- Total = n[log n + (log n − 1) + … + 2 + 1] = n · log n (log n + 1)/2
- This is **Θ(n log² n),** which matches the extended theorem.

(e) Best, Average and Worst Search Cases

S1 – Sequential search

- **Best case:** the target is the first product; 1 comparison.
- **Average successful case:** if the target is equally likely to be at any position, the average is (n+1)/2 comparisons.
- **Worst case:** the target is last or does not exist; n comparisons.
  If the target exists with probability p and its position is uniformly distributed when it exists:
  - E[C] = p(n+1)/2 + (1−p)n = n − p(n−1)/2.

For n=1,000,000 and p=0.5:

- E[C]=1,000,000−0.5(999,999)/2 = 750,000.25 comparisons, approximately 750,000.

S2 – Binary search

- **Best case:** the target is the middle item; 1 comparison.
- **Average case:** approximately $\log_2$n comparisons, depending on the search tree and target distribution.
- **Worst case:** approximately ⌈$\log_2$(n+1)⌉ comparisons. Binary search requires the index to be sorted.

(f) Prove the Report Generator is Θ(n²)

Given: $f(n) = 3n^2 + 10n \log_2 n + 50$

Definition: $f(n) = \Theta(n^2)$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1n^2 \le f(n) \le c_2n^2$ for all $n \ge n_0$.

Lower bound: all terms are positive, so $f(n) \ge 3n^2$. Take $c_1 = 3$.

Upper bound: for $n \ge 1$, we have $\log_2 n \le n$ and $1 \le n^2$.

- $10n \log_2 n \le 10n \cdot n = 10n^2$
- $50 \le 50n^2$

So $f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2$. Take $c_2 = 63$.

Constants: $c_1 = 3$, $c_2 = 63$, $n_0 = 1$.
So $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$. Therefore $f(n) = \Theta(n^2)$.

Proof that f(n) is not O(n):
• Assume $f(n) = O(n)$. Then there are constants c and n_0 with $f(n) \le c \cdot n$ for all $n \ge n_0$.
• Since $f(n) \ge 3n^2$, this gives $3n^2 \le c \cdot n$, which means $n \le c/3$.
• But we can choose n larger than both n_0 and $c/3$ (for example $n = \max(n_0, c/3) + 1$), and this breaks the inequality.
• This is a contradiction, so **f(n) is not O(n)**

(g) Scalability Decision for n = 1,000,000

The server executes 10^8 basic operations per second, and the nightly window is 600 seconds.

Module	Estimated operations	Estimated time	10-minute window
M1	499,999,500,000	4,999.995 s ≈ 83.3 min	Fails
M2	1,048,575	0.0105 s	Passes
S2	About 20–21 comparisons	About 0.00000021 s	Passes
M4 current	About $2n^2-n$ = 1,999,999,000,000*	About 20,000 s ≈ 333 min*	Fails
M4 redesign	$n \log_2 n + n \approx$ 20,931,569*	About 0.209 s*	Passes
M5	$n \cdot k(k+1)/2 + n \approx 2.1 \times 10^8$**, $k \approx \log_2 n$	About 2.1 s*	Passes under this estimate

- Recurrence estimates assume T(1)=1 and use the recurrence-level cost model; exact running times depend on implementation details and constant factors. S2's count is comparisons, not a full catalogue-processing workload.

Recommendations

- Redesign M1 using a hash table keyed by normalized product title and seller ID.
  - This can reduce expected work to O(n). Sorting by the same key and comparing adjacent records is another option with O(n log n) time.
- Adopt the M4 redesign that removes replicated partitions. It changes the complexity from $\Theta(n^2)$ to $\Theta(n \log n)$.
- Use S2 for catalogue searches and maintain a sorted product index.
- M2 can be retained because its operation count is $\Theta(n)$.
- Profile M5 and the nightly report generator with realistic data. The report generator is $\Theta(n^2)$, so optimize it if the full million-record workload threatens the processing window.

Result:

The audit indicates that **M1** and the current **M4** recommendation engine are not suitable for the million-product catalogue under a 10-minute nightly window. **M2** is **$\Theta(n)$, S2** is **$\Theta(\log n)$,** the proposed M4 redesign is **$\Theta(n \log n)$**, and M5 is **$\Theta(n(\log n)^2)$.** The main recommendations are to redesign M1, adopt the non-replicated **M4** design, and benchmark **M5**and the report generator using realistic workloads.

Implementation Output:

```
M1 Duplicate Detector
Operations: 499,999,500,000.00
Time: 4,999.995000 seconds
Decision: Exceeds 10-minute window

M2 API Rate Limiter
Operations: 1,048,575.00
Time: 0.010486 seconds
Decision: Within 10-minute window

S2 Binary Search
Operations: 21.00
Time: 0.000000 seconds
Decision: Within 10-minute window

M4 Current Design (estimate)
Operations: 1,000,000,000,000.00
Time: 10,000.000000 seconds
Decision: Exceeds 10-minute window

M4 Proposed Design (estimate)
Operations: 19,931,568.57
Time: 0.199316 seconds
Decision: Within 10-minute window

M5 Analytics (estimate)
Operations: 208,599,497.10
Time: 2.085995 seconds
Decision: Within 10-minute window

Nightly Report Generator
Operations: 3,000,199,315,735.69
Time: 30,001.993157 seconds
Decision: Exceeds 10-minute window

M2 Verification for n = 1024
Audit operations: 1023


S2 Recurrence
T(n) = T(n/2) + 1
T(1) = 1
Closed form for powers of 2: T(n) = log2(n) + 1
Time complexity: Theta(log n)
Iterative space: Theta(1)
Recursive space: Theta(log n)

Asymptotic Results
M1: Theta(n^2)
M2: Theta(n)
S1: Theta(n) worst case
S2: Theta(log n)
M4 Current: Theta(n^2)
M4 Proposed: Theta(n log n)
M5 Analytics: Theta(n (log n)^2)
Report Generator: Theta(n^2)
```

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	**Weightage (%)**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I **Shaik. Abdul Razak (192524358)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:  Date: **11-10-2026**